

718

标准库占据了新标准文本将近三分之二的篇幅。虽然我们不能详细介绍所有标准库设施，但仍有一些标准库设施在很多应用中都是有用的：tuple、bitset、正则表达式以及随机数。我们还将介绍一些附加的 IO 库功能：格式控制、未格式化 IO 和随机访问。

17.1 tuple 类型

C++
11

tuple 是类似 pair（参见 11.2.3 节，第 379 页）的模板。每个 pair 的成员类型都不相同，但每个 pair 都恰好有两个成员。不同 tuple 类型的成员类型也不相同，但一个 tuple 可以有任意数量的成员。每个确定的 tuple 类型的成员数目是固定的，但一个 tuple 类型的成员数目可以与另一个 tuple 类型不同。

当我们希望将一些数据组合成单一对象，但又不想麻烦地定义一个新数据结构来表示这些数据时，tuple 是非常有用的。表 17.1 列出了 tuple 支持的操作。tuple 类型及其伴随类型和函数都定义在 tuple 头文件中。

表 17.1: tuple 支持的操作	
tuple<T1, T2, ..., Tn> t;	t 是一个 tuple，成员数为 n，第 i 个成员的类型为 Ti。所有成员都进行值初始化（参见 3.3.1 节，第 88 页）
tuple<T1, T2, ..., Tn> t(v1, v2, ..., vn);	t 是一个 tuple，成员类型为 T1...Tn，每个成员用对应的初始值 vi 进行初始化。此构造函数是 explicit 的（参见 7.5.4 节，第 265 页）
make_tuple(v1, v2, ..., vn)	返回一个用给定初始值初始化的 tuple。tuple 的类型从初始值的类型推断
t1 == t2 t1 != t2	当两个 tuple 具有相同数量的成员且成员对应相等时，两个 tuple 相等。这两个操作使用成员的 == 运算符来完成。一旦发现某对成员不等，接下来的成员就不用比较了
t1 relop t2	tuple 的关系运算使用字典序（参见 9.2.7 节，第 304 页）。两个 tuple 必须具有相同数量的成员。使用 < 运算符比较 t1 的成员和 t2 中的对应成员
get<i>(t)	返回 t 的第 i 个数据成员的引用；如果 t 是一个左值，结果是一个左值引用；否则，结果是一个右值引用。tuple 的所有成员都是 public 的
tuple_size<tupleType>::value	一个类模板，可以通过一个 tuple 类型来初始化。它有一个名为 value 的 public constexpr static 数据成员，类型为 size_t，表示给定 tuple 类型中成员的数量
tuple_element<i, tupleType>::type	一个类模板，可以通过一个整型常量和一个 tuple 类型来初始化。它有一个名为 type 的 public 成员，表示给定 tuple 类型中指定成员的类型



我们可以将 tuple 看作一个“快速而随意”的数据结构。

17.1.1 定义和初始化 tuple

当我们定义一个 tuple 时，需要指出每个成员的类型：

```
tuple<size_t, size_t, size_t> threeD; // 三个成员都设置为 0
tuple<string, vector<double>, int, list<int>>
    someVal("constants", {3.14, 2.718}, 42, {0,1,2,3,4,5})
```

当我们创建一个 tuple 对象时，可以使用 tuple 的默认构造函数，它会对每个成员进行值初始化（参见 3.3.1 节，第 88 页）；也可以像本例中初始化 someVal 一样，为每个成员提供一个初始值。tuple 的这个构造函数是 explicit 的（参见 7.5.4 节，第 265 页），因此我们必须使用直接初始化语法：

```
tuple<size_t, size_t, size_t> threeD = {1,2,3}; // 错误
tuple<size_t, size_t, size_t> threeD{1,2,3};    // 正确
```

类似 make_pair 函数（参见 11.2.3 节，第 381 页），标准库定义了 make_tuple 函数，我们还可以用它来生成 tuple 对象：

```
// 表示书店交易记录的 tuple，包含：ISBN、数量和每册书的价格
auto item = make_tuple("0-999-78345-X", 3, 20.00);
```

类似 make_pair，make_tuple 函数使用初始值的类型来推断 tuple 的类型。在本例中，item 是一个 tuple，类型为 tuple<const char*, int, double>。

访问 tuple 的成员

719

一个 pair 总是有两个成员，这样，标准库就可以为它们命名（如，first 和 second）。但这种命名方式对 tuple 是不可能的，因为一个 tuple 类型的成员数目是没有限制的。因此，tuple 的成员都是未命名的。要访问一个 tuple 的成员，就要使用一个名为 get 的标准库函数模板。为了使用 get，我们必须指定一个显式模板实参（参见 16.2.2 节，第 603 页），它指出我们想要访问第几个成员。我们传递给 get 一个 tuple 对象，它返回指定成员的引用：

```
auto book = get<0>(item);           // 返回 item 的第一个成员
auto cnt = get<1>(item);            // 返回 item 的第二个成员
auto price = get<2>(item)/cnt;      // 返回 item 的最后一个成员
get<2>(item) *= 0.8;                // 打折 20%
```

尖括号中的值必须是一个整型常量表达式（参见 2.4.4 节，第 58 页）。与往常一样，我们从 0 开始计数，意味着 get<0>是第一个成员。

如果不知道一个 tuple 准确的类型细节信息，可以用两个辅助类模板来查询 tuple 成员的数量和类型：

```
typedef decltype(item) trans; // trans 是 item 的类型
// 返回 trans 类型对象中成员的数量
size_t sz = tuple_size<trans>::value; // 返回 3
// cnt 的类型与 item 中第二个成员相同
tuple_element<1, trans>::type cnt = get<1>(item); // cnt 是一个 int
```

720

为了使用 tuple_size 或 tuple_element，我们需要知道一个 tuple 对象的类型。与往常一样，确定一个对象的类型的最简单方法就是使用 decltype（参见 2.5.3 节，第 62 页）。在本例中，我们使用 decltype 来为 item 类型定义一个类型别名，用它来实例化

两个模板。

`tuple_size` 有一个名为 `value` 的 `public static` 数据成员，它表示给定 `tuple` 中成员的数量。`tuple_element` 模板除了一个 `tuple` 类型外，还接受一个索引值。它有一个名为 `type` 的 `public` 类型成员，表示给定 `tuple` 类型中指定成员的类型。类似 `get`，`tuple_element` 所使用的索引也是从 0 开始计数的。

关系和相等运算符

`tuple` 的关系和相等运算符的行为类似容器的对应操作（参见 9.2.7 节，第 304 页）。这些运算符逐对比较左侧 `tuple` 和右侧 `tuple` 的成员。只有两个 `tuple` 具有相同数量的成员时，我们才可以比较它们。而且，为了使用 `tuple` 的相等或不等运算符，对每对成员使用 `==` 运算符必须都是合法的；为了使用关系运算符，对每对成员使用 `<` 必须都是合法的。例如：

```
tuple<string, string> duo("1", "2");
tuple<size_t, size_t> twoD(1, 2);
bool b = (duo == twoD); // 错误：不能比较 size_t 和 string
tuple<size_t, size_t, size_t> threeD(1, 2, 3);
b = (twoD < threeD);    // 错误：成员数量不同
tuple<size_t, size_t> origin(0, 0);
b = (origin < twoD);    // 正确：b 为 true
```



由于 `tuple` 定义了 `<` 和 `==` 运算符，我们可以将 `tuple` 序列传递给算法，并且可以在无序容器中将 `tuple` 作为关键字类型。

17.1.1 节练习

练习 17.1： 定义一个保存三个 `int` 值的 `tuple`，并将其成员分别初始化为 10、20 和 30。

练习 17.2： 定义一个 `tuple`，保存一个 `string`、一个 `vector<string>` 和一个 `pair<string, int>`。

练习 17.3： 重写 12.3 节（第 430 页）中的 `TextQuery` 程序，使用 `tuple` 代替 `QueryResult` 类。你认为哪种设计更好？为什么？

721

17.1.2 使用 `tuple` 返回多个值

`tuple` 的一个常见用途是从一个函数返回多个值。例如，我们的书店可能是多家连锁书店中的一家。每家书店都有一个销售记录文件，保存每本书近期的销售数据。我们可能希望在所有书店中查询某本书的销售情况。

假定每家书店都有一个销售记录文件。每个文件都将每本书的所有销售记录存放在一起。进一步假定已有一个函数可以读取这些销售记录文件，为每个书店创建一个 `vector<Sales_data>`，并将这些 `vector` 保存在 `vector` 的 `vector` 中：

```
// files 中的每个元素保存一家书店的销售记录
vector<vector<Sales_data>> files;
```

我们将编写一个函数，对于一本给定的书，在 `files` 中搜索出售过这本书的书店。对每家匹配销售记录的书店，我们将创建一个 `tuple` 来保存这家书店的索引和两个迭代器。

索引指出了书店在 `files` 中的位置，而两个迭代器则标记了给定书籍在此书店的 `vector<Sales_data>` 中第一条销售记录和最后一条销售记录之后的位置。

返回 tuple 的函数

我们首先编写查找给定书籍的函数。此函数的参数是刚刚提到的 `vector` 的 `vector` 以及一个表示书籍 ISBN 的 `string`。我们的函数将返回一个 `tuple` 的 `vector`，凡是销售了给定书籍的书店，都在 `vector` 中有对应的一项：

```
// matches 有三个成员：一家书店的索引和两个指向书店 vector 中元素的迭代器
typedef tuple<vector<Sales_data>::size_type,
            vector<Sales_data>::const_iterator,
            vector<Sales_data>::const_iterator> matches;
// files 保存每家书店的销售记录
// findBook 返回一个 vector，每家销售了给定书籍的书店在其中都有一项
vector<matches>
findBook(const vector<vector<Sales_data>> &files,
        const string &book)
{
    vector<matches> ret; // 初始化为空 vector
    // 对每家书店，查找与给定书籍匹配的记录范围（如果存在的话）
    for (auto it = files.cbegin(); it != files.cend(); ++it) {
        // 查找具有相同 ISBN 的 Sales_data 范围
        auto found = equal_range(it->cbegin(), it->cend(),
                                book, compareIsbn);
        if (found.first != found.second) // 此书店销售了给定书籍
            // 记住此书店的索引及匹配的范围
            ret.push_back(make_tuple(it - files.cbegin(),
                                    found.first, found.second));
    }
    return ret; // 如果未找到匹配记录的话，ret 为空
}
```

`for` 循环遍历 `files` 中的元素，每个元素都是一个 `vector`。在 `for` 循环内，我们调用了一个名为 `equal_range` 的标准库算法，它的功能与关联容器的同名成员类似（参见 11.3.5 节，第 390 页）。`equal_range` 的前两个实参是表示输入序列的迭代器（参见 10.1 节，第 336 页），第三个参数是一个值。默认情况下，`equal_range` 使用 `<` 运算符来比较元素。由于 `Sales_data` 没有 `<` 运算符，因此我们传递给它一个指向 `compareIsbn` 函数的指针（参见 11.2.2 节，第 379 页）。 722

`equal_range` 算法返回一个迭代器 `pair`，表示元素的范围。如果未找到 `book`，则两个迭代器相等，表示空范围。否则，返回的 `pair` 的 `first` 成员将表示第一条匹配的记录，`second` 则表示匹配的尾后位置。

使用函数返回的 tuple

一旦我们创建了 `vector` 保存包含匹配的销售记录的书店，就需要处理这些记录了。在此程序中，对每家包含匹配销售记录的书店，我们将打印其汇总销售信息：

```
void reportResults(istream &in, ostream &os,
                 const vector<vector<Sales_data>> &files)
{
    string s; // 要查找的书
```

```

while (in >> s) {
    auto trans = findBook(files, s); // 销售了这本书的书店
    if (trans.empty()) {
        cout << s << " not found in any stores" << endl;
        continue; // 获得下一本要查找的书
    }
    for (const auto &store : trans) // 对每家销售了给定书籍的书店
        // get<n>返回 store 中 tuple 的指定的成员
        os << "store " << get<0>(store) << " sales: "
            << accumulate(get<1>(store), get<2>(store),
                           Sales_data(s))
            << endl;
    }
}

```

while 循环反复读取名为 in 的 istream 来获得下一本要处理的书。我们调用 findBook 来检查 s 是否存在，并将结果赋予 trans。我们使用 auto 来简化 trans 类型的代码编写，它是一个 tuple 的 vector。

如果 trans 为空，表示没有关于 s 的销售记录。在此情况下，我们打印一条信息并返回，执行下一步 while 循环来获取下一本要查找的书。

for 循环将 store 绑定到 trans 中的每个元素。由于不希望改变 trans 中的元素，我们将 store 声明为 const 的引用。我们使用 get 来打印相关数据：get<0>表示对应书店的索引、get<1>表示第一条交易记录的迭代器、get<2>表示尾后位置的迭代器。

由于 Sales_data 定义了加法运算符（参见 14.3 节，第 497 页），因此我们可以用标准库的 accumulate 算法（参见 10.2.1 节，第 338 页）来累加销售记录。我们用 Sales_data 的接受一个 string 参数的构造函数（参见 7.1.4 节，第 236 页）来初始化一个 Sales_data 对象，将此对象传递给 accumulate 作为求和的起点。此构造函数用给定的 string 初始化 bookNo，并将 units_sold 和 revenue 成员置为 0。

723

17.1.2 节练习

练习 17.4：编写并测试你自己版本的 findBook 函数。

练习 17.5：重写 findBook，令其返回一个 pair，包含一个索引和一个迭代器 pair。

练习 17.6：重写 findBook，不使用 tuple 或 pair。

练习 17.7：解释你更倾向于哪个版本的 findBook，为什么。

练习 17.8：在本节最后一段代码中，如果我们将 Sales_data() 作为第三个参数传递给 accumulate，会发生什么？

17.2 bitset 类型

在 4.8 节（第 135 页）中我们介绍了将整型运算对象当作二进制位集合处理的一些内置运算符。标准库还定义了 **bitset** 类，使得位运算的使用更为容易，并且能够处理超过最长整型类型大小的位集合。bitset 类定义在头文件 `bitset` 中。